

Build a Local AI Email Auto-Responder with n8n and Ollama

Bonus (ai)

Back

Domain

Backend Development

Machine Learning

Skills

AI App Development

API Development

Automation

Configuration Management

Prompt Engineering

System Design

Difficulty

Intermediate

Tools

Docker

Git

Llama

N8N

Industries

Technology



Overdue - Late Submission Allowed

OVERDUE BY

30d

The deadline passed 30d ago. However, submission evaluation has not started yet. You can still submit, but it will be marked as late and may be subject to penalties.

Deadline: 2 May 2026, 04:59 pm

Overview

Instructions

Resources

Submit

Objective



Report Issue

Build a fully local, private AI email auto-responder using n8n for workflow automation and Ollama for on-device LLM inference. You

partnr

Dashboard Skill Graph Profile GPP Job Applications

without cloud dependencies, a crucial skill in an era of data sensitivity and rising AI costs.

Description

1. Project Overview and Background

Welcome to a hands-on project at the intersection of workflow automation, artificial intelligence, and data privacy. You will build an intelligent email auto-responder that runs entirely on your local machine. Unlike solutions that rely on third-party APIs like OpenAI or Google, this system ensures that your data (in this case, your emails) never leaves your environment. This is achieved by combining two powerful open-source tools: n8n for workflow orchestration and Ollama for running large language models (LLMs) locally.

The core idea is to create a system that can read an incoming email, understand its content and intent, and generate a human-like, contextually relevant reply. This is not a simple "out-of-office" reply; it's a smart agent capable of handling initial queries, categorizing requests, or even answering simple questions, all powered by a local LLM. This project will teach you how to build practical AI applications that are cost-effective, private, and completely under your control.

Why Local AI?

1. **Privacy:** Your data is never sent to a third-party service. This is critical for applications handling sensitive or personal information.
2. **Cost:** Running local models eliminates API call costs, which can become significant at scale. The only cost is the hardware you already own.
3. **Control & Customization:** You have full control over the model, the prompts, and the entire system's logic. You can fine-tune models for specific tasks without being constrained by a provider's terms of service.
4. **Offline Capability:** The system can operate without an internet connection (aside from fetching the emails themselves).

2. Core Technologies Deep Dive

To succeed in this project, you must understand the role of each component. Let's break them down.

n8n: The Automation Engine

partnr

Dashboard Skill Graph Profile GPP Job Applications

1. **Nodes:** Each node in n8n represents a specific action or application. For this project, you'll use nodes for reading emails (IMAP), making HTTP requests (to Ollama), implementing conditional logic (IF), and sending emails (SMTP).
2. **Workflows:** A workflow is a collection of connected nodes that define a sequence of operations. Our entire auto-responder logic will be encapsulated in a single n8n workflow.
3. **Triggers:** A trigger is a special node that starts a workflow. Our trigger will be the "Email Read (IMAP)" node, which will activate whenever a new email arrives.
4. **Credentials:** n8n securely stores credentials (like email passwords or API keys) so you don't have to hardcode them into your workflow.

Ollama: Your Local LLM Server

Ollama is a tool that dramatically simplifies the process of downloading and running state-of-the-art open-source LLMs like Llama 3, Mistral, and Phi-3. It packages these models into a ready-to-use service and exposes a simple REST API for interacting with them.

1. **Local Inference:** Ollama handles all the complexities of setting up the model and running inference on your local hardware (CPU or GPU).
2. **API Endpoint:** It provides a consistent API endpoint (typically `http://localhost:11434/api/generate`) that we can call from our n8n workflow to get a response from the LLM. This decouples the AI model from our automation logic.
3. **Model Management:** You can easily pull different models from the Ollama library using a single command (e.g., `ollama pull llama3:8b`).

Docker and Docker Compose: The Environment Manager

Since our system consists of multiple services (n8n and Ollama) that need to run and communicate with each other, we need a way to manage them effectively. Docker and Docker Compose are the industry standard for this.

1. **Docker:** A platform for developing, shipping, and running applications in containers. A container packages up an application's code along with all its dependencies, ensuring it runs consistently across different environments.

2. **Docker Compose:** A tool for defining and running multi-container Docker applications. We will use a `docker-compose.yml` file to

partnr

Dashboard

Skill Graph

Profile

GPP

Job Applications

3. System Architecture and Design

The architecture is a straightforward pipeline that processes emails. The data flows from your email server, through the n8n workflow for processing and enrichment via Ollama, and back to the email server to send the reply.

Here is a diagram illustrating the complete data flow:

Expand

partnr

Dashboard Skill Graph Profile GPP Job Applications

partnr

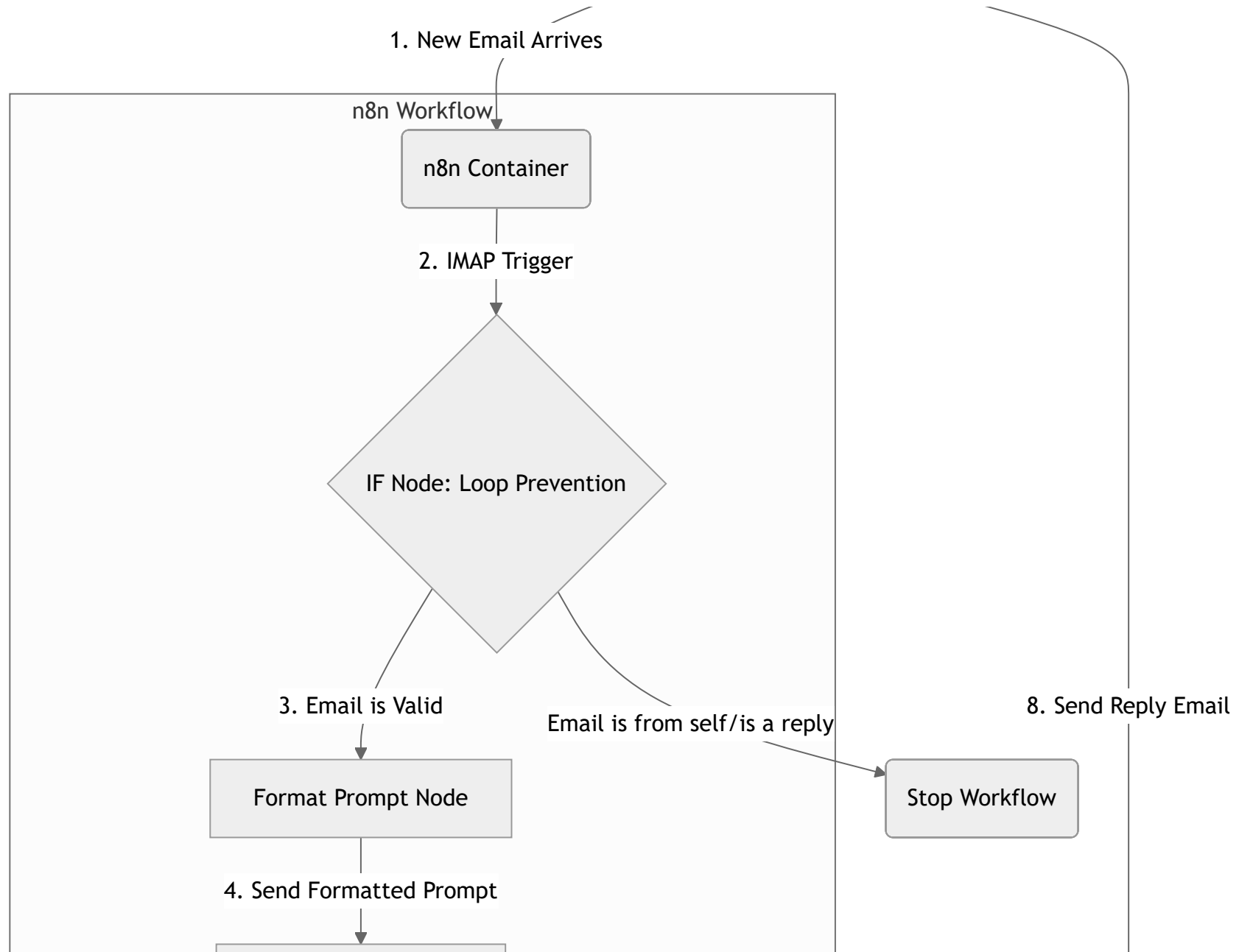
Dashboard

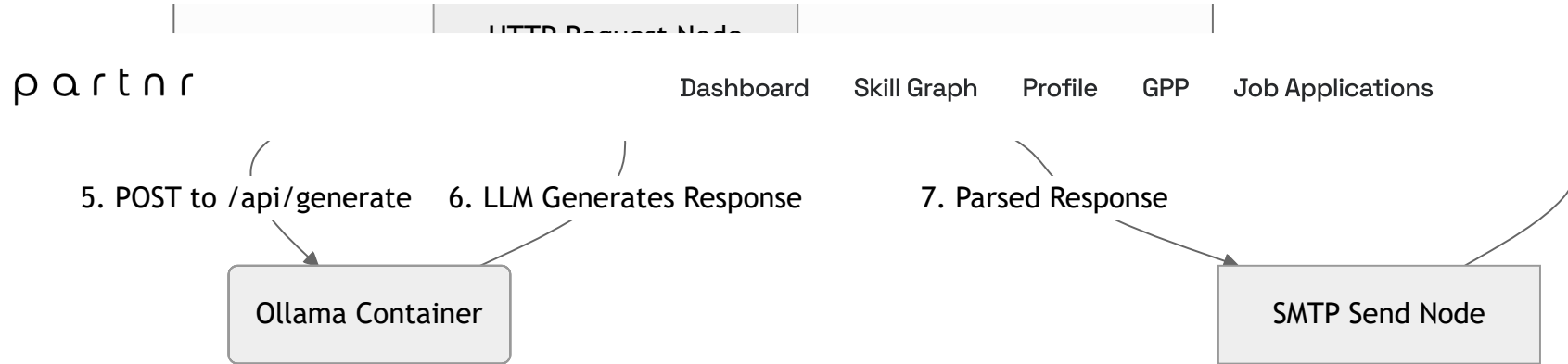
Skill Graph

Profile

GPP

Job Applications





Detailed Data Flow Breakdown:

- Email Arrival:** An external email is sent to the address you've configured.
- n8n IMAP Trigger:** The n8n workflow, which is constantly polling the IMAP server, detects the new email and starts an execution. It ingests the email's data, including sender, subject, body, and message ID.
- Loop Prevention:** The first crucial step is an IF node that checks if the email should be processed. It might check if the sender is the same as the auto-responder's address or if the subject line already contains our auto-reply tag (e.g., "Re: AI Auto-Reply:"). This prevents infinite loops where your bot replies to its own replies.
- Prompt Formatting:** A node (e.g., a "Set" or "Code" node) takes the raw subject and body of the email and formats it into a clear, structured prompt for the LLM. This is a critical step in getting good responses. A good prompt might look like this:

You are a professional and helpful email assistant. A new email has arrived. Please write a concise and relevant reply.

```
Sender: {{ sender_email }}
Subject: {{ email_subject }}
Body:
{{ email_body }}
```

Your reply should be helpful and directly address the content of the email. Do not include a greeting or signature; provide only the

- Ollama API Call:** The HTTP Request node sends this formatted prompt in a POST request to the Ollama service's `/api/generate` endpoint. Since both services are in the same Docker network, n8n can address Ollama directly by its service name (e.g., `http://ollama:11434/api/generate`).
- LLM Inference:** Ollama receives the request, passes the prompt to the loaded LLM (e.g., `llama3:8b`), and generates a response.

7. Response Parsing & Sending: The HTTP Request node in n8n receives the JSON response from Ollama. A subsequent node

partnr

Dashboard Skill Graph Profile GPP Job Applications

same conversation thread in email clients.

8. Workflow Completion: The workflow execution is logged as successful in the n8n UI.

4. Step-by-Step Implementation Guide

Follow these steps to build the system from the ground up.

Step 1: Setting Up the Docker Environment

Your first task is to create the `docker-compose.yml` file. This file will be the foundation of your project.

```
# docker-compose.yml (example structure)
version: '3.8'

services:
  n8n:
    image: n8nio/n8n
    container_name: n8n_responder
    ports:
      - "5678:5678"
    environment:
      - N8N_HOST=localhost
      - N8N_PORT=5678
      - N8N_PROTOCOL=http
      # Add other n8n env vars as needed
    volumes:
      - n8n_data:/home/node/.n8n
    networks:
      - ai_responder_net
    healthcheck:
      test: ["CMD-SHELL", "wget -q --spider --proxy=off http://localhost:5678/healthz || exit 1"]
      interval: 30s
      timeout: 10s
      retries: 3
```


ollama:

partnr

Dashboard Skill Graph Profile GPP Job Applications

```
- ollama_data:/root/.ollama
networks:
- ai_responder_net
# If you have an Nvidia GPU, you can add deployment configs here
healthcheck:
  test: ["CMD", "ollama", "ps"]
  interval: 30s
  timeout: 10s
  retries: 3
```

```
volumes:
  n8n_data:
  ollama_data:
```

```
networks:
  ai_responder_net:
```

Why this structure?

1. **Services:** We define `n8n` and `ollama` as two distinct services.
2. **Volumes:** We use named volumes (`n8n_data` , `ollama_data`) to persist data. This ensures your n8n workflows and downloaded Ollama models are not lost when you restart the containers.
3. **Network:** We create a custom bridge network (`ai_responder_net`) so the containers can communicate with each other using their service names as hostnames (e.g., `http://ollama:11434`).
4. **Healthchecks:** These are crucial for ensuring that one service doesn't try to communicate with another before it's ready.

Step 2: Configuring and Testing Ollama

Once your `docker-compose.yml` is ready, start the services:

```
docker-compose up -d
```

After the containers are running, you need to pull an LLM into your Ollama instance. You can do this by executing a command

partnr

Dashboard Skill Graph Profile GPP Job Applications

```
docker exec -it ollama_responder ollama pull llama3:8b
```

To verify that Ollama is working and the model is available, you can send a `curl` request to its API from your host machine (since you haven't exposed its port, you'll do this from within the n8n container later, but for an initial check, you can expose the port 11434:11434 temporarily).

Step 3: Building the n8n Workflow

Navigate to `http://localhost:5678` in your browser to access the n8n UI. Create a new, blank workflow.

- Add the IMAP Trigger:** Search for the "Email Read (IMAP)" node and add it to your canvas. You will need to create new credentials for it, providing your email server's IMAP host, port, username, and password. Configure it to trigger for "Unread" messages in your "INBOX".
- Add the Loop Prevention IF Node:** Add an "IF" node and connect it to the IMAP trigger. Set up a condition to check if the sender's email address is the same as your own. You can use an expression like `{{ $json.from.address }}`. You can add another `OR` condition to check if the subject starts with your reply prefix.
- Add the HTTP Request Node for Ollama:** Add an "HTTP Request" node. Connect it to the `true` output of the IF node.
 - URL:** `http://ollama:11434/api/generate`
 - Method:** `POST`
 - Body Content Type:** `JSON`
 - Body:** This is where you construct the JSON payload for Ollama. Click "Add Expression" for the body and create a structure like this:

```
{
  "model": "llama3:8b",
  "stream": false,
  "prompt": "You are a helpful assistant. A new email has arrived from {{ $json.from.address }}. The subject is '{{ $json.subject }}'."
}
```

4. **Add the SMTP Send Node:** Add a "Send Email (SMTP)" node. Connect it to the HTTP Request node. Configure its credentials

partnr

Dashboard Skill Graph Profile GPP Job Applications

2. **Subject:** Prepend `Re:` to the original subject: `Re: {{ $json.subject }}`.
3. **Text:** This is where you use the output from the Ollama node. The expression will look something like this: `{{ $node["HTTP Request"].json.response }}`. You need to get the `response` field from the JSON returned by Ollama.
4. **Crucial Step – Threading:** Go to "Options" -> "Add Option" -> "Message To Reply To". For the "Message-ID" field, use an expression to pass the ID of the original email: `{{ $json.messageId }}`. This is the key to maintaining email threads.

Step 4: Finalizing and Exporting

Activate your workflow in the n8n UI. Once you are satisfied with its functionality, you must export it. Click the "Download" button in the workflow editor to save the workflow as a `workflow.json` file. Place this file in the root of your project repository.

5. Testing and Verification

To test your system end-to-end:

1. Ensure your Docker containers are running and healthy (`docker-compose ps`).
2. Send an email from a different account to the email address you configured in the n8n IMAP trigger.
3. Go to the n8n UI and look at "Executions". You should see a new workflow execution start.
4. Click on the execution to inspect the data flow. Check the output of each node to ensure the data is being passed and transformed correctly.
5. Check the inbox of the account you sent the email from. You should receive a reply within a minute or two.
6. Verify that the reply appears in the same conversation thread as the original email.
7. Send another email from your auto-responder's own address to itself. Verify that the workflow is triggered but stops at the IF node, preventing a loop.

6. FAQ

Q: My n8n container can't connect to the Ollama container. What's wrong?

A: This is almost always a Docker networking issue. Ensure both containers are on the same custom network in your `docker-`

partnr

Dashboard Skill Graph Profile GPP Job Applications

Q: Ollama is running, but the API call fails with a model not found error.

A: Make sure you have successfully pulled the model into the Ollama instance running inside the container. Use `docker exec -it ollama_responder ollama list` to see which models are available. The model name in your HTTP request body (`"model": "llama3:8b"`) must exactly match the name of a model in that list.

Q: How do I handle sensitive information like my email password?

A: Do not hardcode credentials in your `docker-compose.yml` or `workflow.json` . Use environment variables. n8n is designed to read credentials from environment variables. Define them in a `.env` file (which you should add to `.gitignore`) and reference them in your `docker-compose.yml` . Your submitted `.env.example` file should list the variable names but with placeholder values.

Q: The LLM response is not what I expected. How can I improve it?

A: This is where prompt engineering comes in. The quality of your output is highly dependent on the quality of your input prompt. Be very specific in your instructions to the model in the HTTP Request node. Tell it about its persona ("You are a professional assistant"), the context, and the exact format you want the output in ("Only provide the reply body"). Experiment with different prompts to get the desired result.

Core Requirements

1. The project must contain a `docker-compose.yml` file at the root that defines and configures the `n8n` and `ollama` services.

Contract Specification

1. **File Location:** `docker-compose.yml`
2. **Required Services:**
 1. `n8n` : Must use a valid n8n image (e.g., `n8nio/n8n`).
 2. `ollama` : Must use a valid Ollama image (e.g., `ollama/ollama`).
3. **Networking:** Both services must be connected to the same custom bridge network.

4. **Health Checks:** Both the `n8n` and `ollama` services must have a `healthcheck` defined to ensure they are running correctly.

partnr

Dashboard Skill Graph Profile GPP Job Applications

Verification

1. Run `docker-compose up -d --build`.
2. Verify that two containers, one for `n8n` and one for `ollama`, are created and running.
3. Run `docker-compose ps` and verify that the status of both containers becomes `healthy` within 3 minutes.

-
2. The Ollama service must be configured to automatically have a specific LLM available for inference upon startup. The recommended model is `llama3:8b`.

Contract Specification

1. **Model Name:** `llama3:8b` (or another small, suitable model specified in the README).
2. **Configuration:** The model should be pulled either via a startup script, a custom Dockerfile that extends the base Ollama image, or clear instructions in the README on the single command to run after `docker-compose up`.

Verification

1. After the services are up and healthy, execute a command to check the Ollama API for available models.
2. Send a `GET` request to `http://<ollama_container_ip>:11434/api/tags`.
3. Verify that the JSON response contains an object where the `name` field matches the required model (e.g., `llama3:8b`).

-
3. The complete and functional `n8n` workflow must be exported and included in the repository.

Contract Specification

1. **File Location:** `workflow.json` in the root of the repository.
2. **Format:** The file must be a valid JSON export from `n8n`, containing the nodes, connections, and credentials configuration for the workflow.

Verification

partnr

Dashboard Skill Graph Profile GPP Job Applications

3. Verify the JSON contains top-level keys such as `name`, `nodes`, and `connections`.

4. The n8n workflow must be triggered by new emails using the IMAP protocol.

Contract Specification

1. **Node Type:** The workflow must start with a trigger node of type `n8n-nodes-base.emailReadImap`.
2. **Configuration:** The node should be configured to check a specific IMAP folder (e.g., `INBOX`) for unread messages.

Verification

1. Parse the `workflow.json` file.
 2. Find the node object where the `type` property is `n8n-nodes-base.emailReadImap`.
 3. Verify this node is a trigger (has no nodes connecting to its input).
-

5. The workflow must make an HTTP POST request to the local Ollama service to generate a response.

Contract Specification

1. **Node Type:** The workflow must contain a node of type `n8n-nodes-base.httpRequest`.
2. **Request Details:**
 1. **Method:** `POST`
 2. **URL:** `http://ollama:11434/api/generate` (must use the service name `ollama`, not `localhost` or an IP address).
 3. **Body:** The request body must be JSON and contain `model` and `prompt` keys.

Verification

1. Parse `workflow.json` and find the `n8n-nodes-base.httpRequest` node.

2. Verify its `parameters` object contains the correct `method`, `url`, and a `body` structure that sends a JSON object.

partnr

Dashboard

Skill Graph

Profile

GPP

Job Applications

Contract Specification

1. **Prompt Content:** The `prompt` string sent in the body of the HTTP request to Ollama must include expressions that reference the subject and body of the triggering email.
2. **Example Expressions:** The prompt should contain placeholders like `{{ $json.subject }}` and `{{ $json.text }}` or `{{ $json.html }}`.

Verification

1. Parse `workflow.json` and inspect the `body` parameter of the Ollama HTTP request node.
2. Verify that the `prompt` value is an expression string containing references to properties of an earlier node's JSON output (e.g., `$json.subject`).

7. The workflow must correctly parse the response from the Ollama API to extract the generated text.

Contract Specification

1. **Ollama Response Format:** The Ollama API (with `stream: false`) returns a JSON object like `{"response": "..."}.`
2. **Workflow Logic:** A node subsequent to the Ollama HTTP request must reference the `response` field from the output of that request.

Verification

1. Parse `workflow.json`.
2. Identify the node that sends the email (e.g., `n8n-nodes-base.emailSend`).
3. Check the parameter for the email body (e.g., `text`). Its value must be an expression that references the output of the HTTP request node, specifically targeting the `response` key (e.g., `{{ $node["HTTP Request"].json.response }}`).

8. The workflow must send the AI-generated response as an email using the SMTP protocol.

partnr

Dashboard Skill Graph Profile GPP Job Applications

1. **Node Type:** The workflow must contain a node of type `n8n-nodes-base.emailSend`.
2. **Recipient:** The `to` field must be dynamically set to the sender of the original email.

Verification

1. Parse `workflow.json` and find the `n8n-nodes-base.emailSend` node.
2. Verify its `parameters.to` field is set with an expression referencing the original sender's address (e.g., `{{ $json.from.address }}`).

9. The sent email reply must correctly maintain the email conversation thread.

Contract Specification

1. **Email Headers:** The reply email must include the `In-Reply-To` and `References` headers, populated with the `Message-ID` of the original email.
2. **n8n Configuration:** In the "Send Email" node, the "Message To Reply To" -> "Message-ID" option must be set.

Verification

1. Parse `workflow.json` and find the `n8n-nodes-base.emailSend` node.
2. Inspect its `parameters.options.messageId` field.
3. Verify this field is set to an expression that dynamically uses the message ID of the trigger email (e.g., `{{ $json.messageId }}`).

10. The workflow must include logic to prevent it from replying to its own messages, avoiding infinite loops.

Contract Specification

1. **Node Type:** The workflow must contain a logic node, such as `n8n-nodes-base.if`.

2. **Condition:** This node must execute after the triaquer and before the Ollama request. It must check at least one of the

partnr

Dashboard Skill Graph Profile GPP Job Applications

2. If the email subject contains a specific auto-reply indicator (e.g., starts with `Re: AI Auto-Reply:`).

3. **Workflow Path:** The Ollama request and email sending nodes must only be on the `true` path of this conditional logic.

Verification

1. Parse `workflow.json` and locate the `n8n-nodes-base.if` node.
2. Analyze its `parameters.conditions` to confirm it checks the sender or subject.
3. Analyze the `connections` data to ensure the HTTP Request node is connected to the `true` output (`output_0`) of the IF node.

11. An `.env.example` file must be present to document all necessary environment variables.

Contract Specification

1. **File Location:** `.env.example` in the repository root.
2. **Content:** Must include placeholder variables for all required credentials and configurations, such as:
 1. IMAP Host, User, Password, Port
 2. SMTP Host, User, Password, Port
 3. Any n8n-specific environment variables used (e.g., `WEBHOOK_URL`).

Verification

1. Check for the existence of `.env.example` .
2. Review its contents to ensure all necessary configuration variables are documented with placeholder values.

12. A `submission.json` file must be provided with test email credentials for automated evaluation.

Contract Specification

1. **File Location:** `submission.json` in the repository root.

2. Required Structure:

partnr

[Dashboard](#)[Skill Graph](#)[Profile](#)[GPP](#)[Job Applications](#)

```
"imap": {  
  "host": "string",  
  "port": "number",  
  "user": "string",  
  "password": "string"  
},  
"smtp": {  
  "host": "string",  
  "port": "number",  
  "user": "string",  
  "password": "string"  
}  
}  
}
```

Verification

1. Check for the existence of `submission.json`.
2. Parse the file as JSON and validate its structure against the specified schema.

[About Us](#) [Contact Us](#) [Privacy Policy](#) [Terms and Conditions](#)



partnr

Dashboard Skill Graph Profile GPP Job Applications